Solidity By Example

Solidity is a variant of JavaScript adapted for Ethereum. The examples below highlight the main features of Solidity and show how it works.

Counter Example

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Counter {
    uint256 public count;

    // Function to get the current count
    function get() public view returns (uint256) {
        return count;
    }

    // Function to increment count by 1
    function inc() public {
        count += 1;
    }

    // Function to decrement count by 1
    function dec() public {
        // This function will fail if count = 0
        count -= 1;
    }
}
```

Primitive Data Types:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Primitives {
    bool public boo = true;

    /*
    uint stands for unsigned integer, meaning non negative integers
```

```
different sizes are available
   uint8   ranges from 0 to 2 ** 8 - 1
   uint16  ranges from 0 to 2 ** 16 - 1
   ...
   uint256 ranges from 0 to 2 ** 256 - 1
*/
uint8 public u8 = 1;
uint256 public u256 = 456;
uint256 public u = 123; // uint is an alias for uint256

/*
Negative numbers are allowed for int types.
Like uint, different ranges are available from int8 to int256

int256 ranges from -2 ** 255 to 2 ** 255 - 1
int128 ranges from -2 ** 127 to 2 ** 127 - 1
*/
int8 public i8 = -1;
int256 public i256 = 456;
int256 public i = -123; // int is same as int256

// minimum and maximum of int
int256 public minInt = type(int256).min;
int256 public maxInt = type(int256).max;

address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;

/*
In Solidity, the data type byte represent a sequence of bytes.
Solidity presents two types of bytes :

 - fixed-sized byte arrays
 - dynamically-sized byte arrays.

 The term bytes in Solidity represents a dynamic array of bytes.
 It’s a shorthand for byte[] .
*/
bytes1 a = 0xb5; //  [10110101]
bytes1 b = 0x56; //  [01010110]

// Default values
// Unassigned variables have a default value
bool public defaultBoo; // false
uint256 public defaultUint; // 0
```

```
    int256 public defaultInt; // 0
    address public defaultAddr; // 0x0000000000000000000000000000000000000000
}
```

Variables:

There are 3 types of variables in Solidity

- **local**
    - declared inside a function
    - not stored on the blockchain
- **state**
    - declared outside a function
    - stored on the blockchain
- **global** (provides information about the blockchain)

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.26;

contract Variables {
    // State variables are stored on the blockchain.
    string public text = "Hello";
    uint256 public num = 123;

    function doSomething() public view {
        // Local variables are not saved to the blockchain.
        uint256 i = 456;

        // Here are some global variables
        uint256 timestamp = block.timestamp; // Current block timestamp
        address sender = msg.sender; // address of the caller
    }
}
```

Constants:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Constants {
    // coding convention to uppercase constant variables
    address public constant MY_ADDRESS =
        0x777788889999AaAAbBbbCcccddDdeeeEfFFfCcCc;
    uint256 public constant MY_UINT = 123;
}
```

Immutables:

Immutable variables are like constants. Values of immutable variables can be set inside the constructor but cannot be modified afterwards.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Immutable {
    address public immutable myAddr;
    uint256 public immutable myUint;

    constructor(uint256 _myUint) {
        myAddr = msg.sender;
        myUint = _myUint;
    }
}
```

Reading and Writing to State Variables:

To write or update a state variable you need to send a transaction.

On the other hand, you can read state variables, for free, without any transaction fee.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract SimpleStorage {
    // State variable to store a number
    uint256 public num;

    // You need to send a transaction to write to a state variable.
    function set(uint256 _num) public {
        num = _num;
    }

    // You can read from a state variable without sending a transaction.
    function get() public view returns (uint256) {
        return num;
    }
}
```

Ether and Wei:

Transactions are paid with ether.

Similar to how one dollar is equal to 100 cents, one ether is equal to 10^{18} wei.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract EtherUnits {
    uint256 public oneWei = 1 wei;
    // 1 wei is equal to 1
    bool public isOneWei = (oneWei == 1);

    uint256 public oneGwei = 1 gwei;
    // 1 gwei is equal to 10^9 wei
    bool public isOneGwei = (oneGwei == 1e9);

    uint256 public oneEther = 1 ether;
    // 1 ether is equal to 10^18 wei
    bool public isOneEther = (oneEther == 1e18);
}
```

Gas and Gas Price:

How much ether do you need to pay for a transaction?

You pay gas spent * gas price amount of ether, where

- gas is a unit of computation
- gas spent is the total amount of gas used in a transaction
- gas price is how much ether you are willing to pay per gas

Transactions with higher gas price have higher priority to be included in a block.

Unspent gas will be refunded.

Gas Limit

There are 2 upper bounds to the amount of gas you can spend

- gas limit (max amount of gas you're willing to use for your transaction, set by you)
- block gas limit (max amount of gas allowed in a block, set by the network)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Gas {
    uint256 public i = 0;

    // Using up all of the gas that you send causes your transaction to fail.
    // State changes are undone.
    // Gas spent is not refunded.
    function forever() public {
        // Here we run a loop until all of the gas are spent
        // and the transaction fails
        while (true) {
            i += 1;
        }
    }
}
```

If / else

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;
```

```
contract IfElse {
    function foo(uint256 x) public pure returns (uint256) {
        if (x < 10) {
            return 0;
        } else if (x < 20) {
            return 1;
        } else {
            return 2;
        }
    }

    function ternary(uint256 _x) public pure returns (uint256) {
        // if (_x < 10) {
        //     return 1;
        // }
        // return 2;

        // shorthand way to write if / else statement
        // the "?" operator is called the ternary operator
        return _x < 10 ? 1 : 2;
    }
}
```

for /while loops

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Loop {
    function loop() public pure {
        // for loop
        for (uint256 i = 0; i < 10; i++) {
            if (i == 3) {
                // Skip to next iteration with continue
                continue;
            }
            if (i == 5) {
                // Exit loop with break
                break;
            }
        }
```

```
        // while loop
        uint256 j;
        while (j < 10) {
            j++;
        }
    }
}
```

Mappings:

Maps are created with the syntax mapping(keyType => valueType).
The keyType can be any built-in value type, bytes, string, or any contract.
valueType can be any type including another mapping or an array.
Mappings are not iterable.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Mapping {
    // Mapping from address to uint
    mapping(address => uint256) public myMap;

    function get(address _addr) public view returns (uint256) {
        // Mapping always returns a value.
        // If the value was never set, it will return the default value.
        return myMap[_addr];
    }

    function set(address _addr, uint256 _i) public {
        // Update the value at this address
        myMap[_addr] = _i;
    }

    function remove(address _addr) public {
        // Reset the value to the default value.
        delete myMap[_addr];
    }
}
```

```
contract NestedMapping {
    // Nested mapping (mapping from address to another mapping)
    mapping(address => mapping(uint256 => bool)) public nested;

    function get(address _addr1, uint256 _i) public view returns (bool) {
        // You can get values from a nested mapping
        // even when it is not initialized
        return nested[_addr1][_i];
    }

    function set(address _addr1, uint256 _i, bool _boo) public {
        nested[_addr1][_i] = _boo;
    }

    function remove(address _addr1, uint256 _i) public {
        delete nested[_addr1][_i];
    }
}
```